FUNDAÇÃO EDSON QUEIROZ
UNIVERSIDADE DE FORTALEZA
ENSINANDO E APRENDENDO

Sistemas Distribuídos

Professor: Américo Tadeu Falcone Sampaio

Roteiro

- *Microservices*
  - *O que são*
  - *Aplicação Monolítica Versus Microservice*
  - *Microservices VS SOA*
  - *Características*
  - *Cuidados ao utilizar*
  - *Case studies*
  - *Exemplos de aplicações*
  - *Deployment strategies*

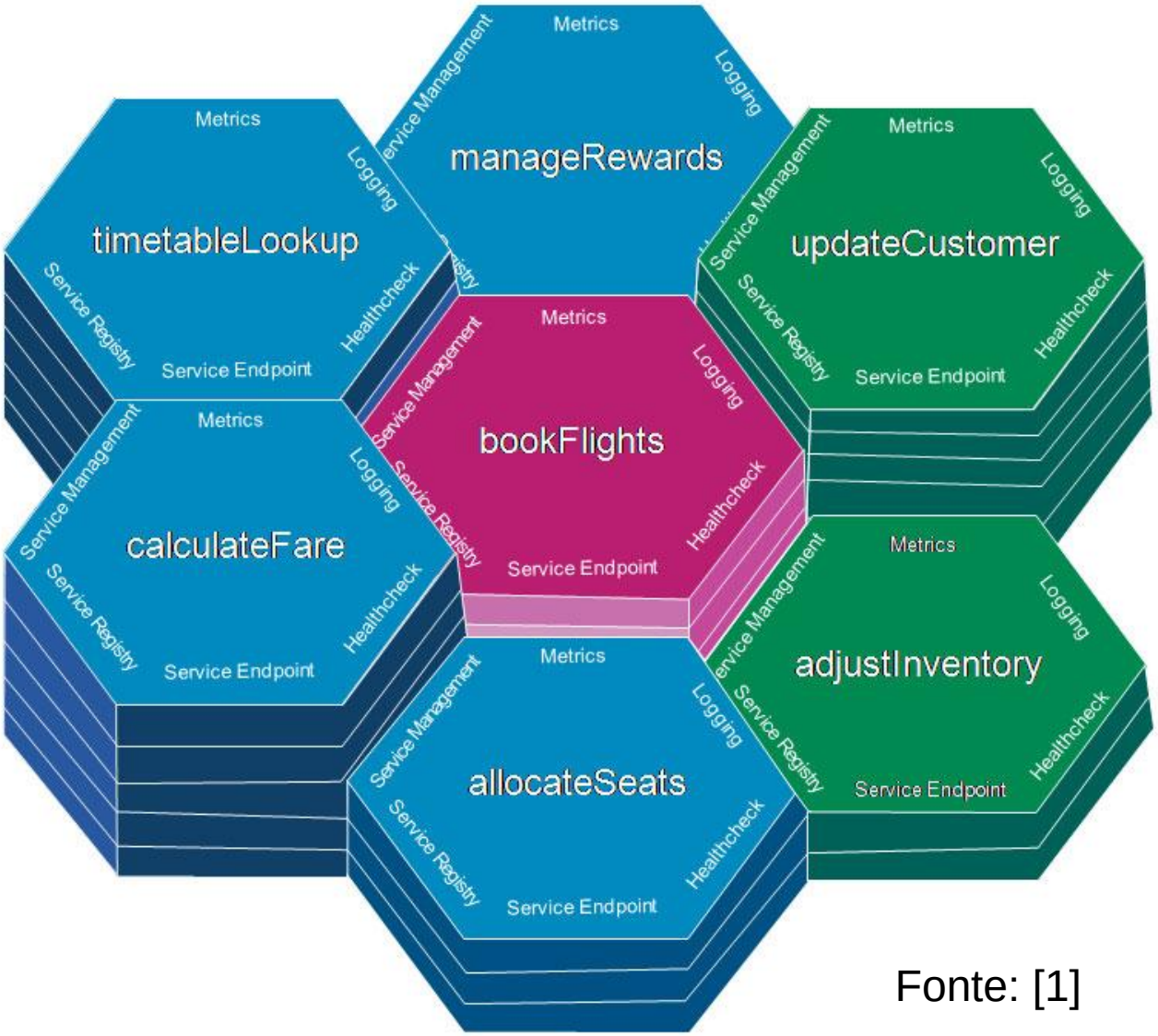
O que são microservices?

- *"Microservices is an architecture style, in which large complex software applications are composed of one or more services. Microservice can be deployed independently of one another and are loosely coupled. Each of these microservices focuses on completing one task only and does that one task really well. In all cases, that one task represents a small business capability." [1]*
  - *Definição bastante similar a um serviço de SOA. Em breve discutiremos as diferenças.*

O que são microservices?

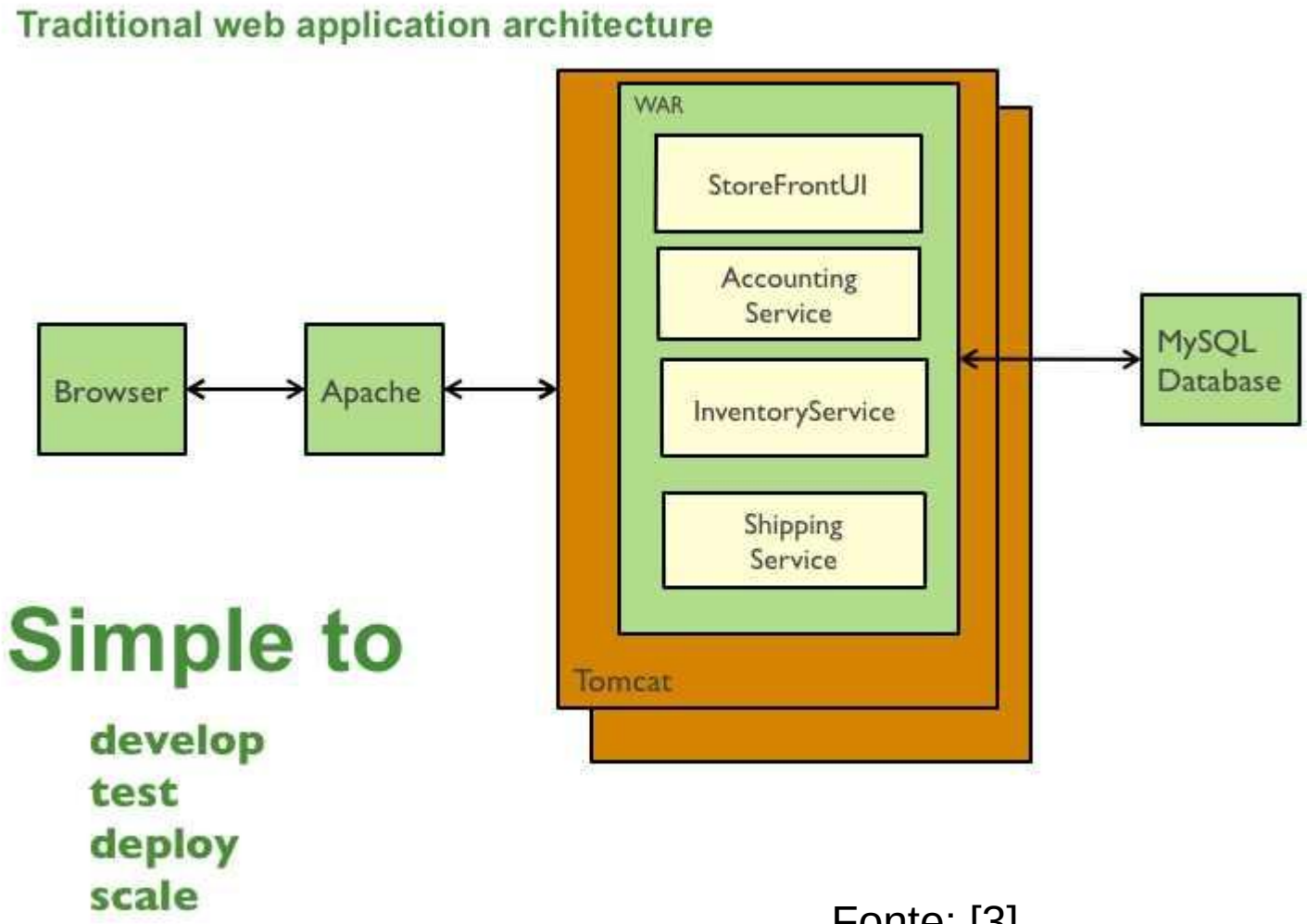
- *"In short, the microservice architectural style is an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API. These services are built around business capabilities and independently deployable by fully automated deployment machinery. There is a bare minimum of centralized management of these services, which may be written in different programming languages and use different data storage technologies." [4,5]*

Exemplo de Aplicação que usa Microservice



Fonte: [1]

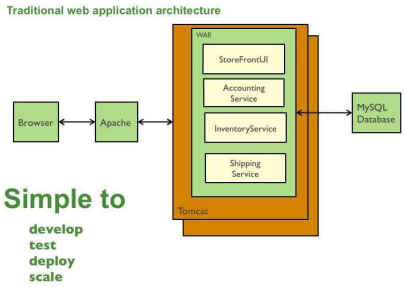
Exemplo 2 – Monolithic Version



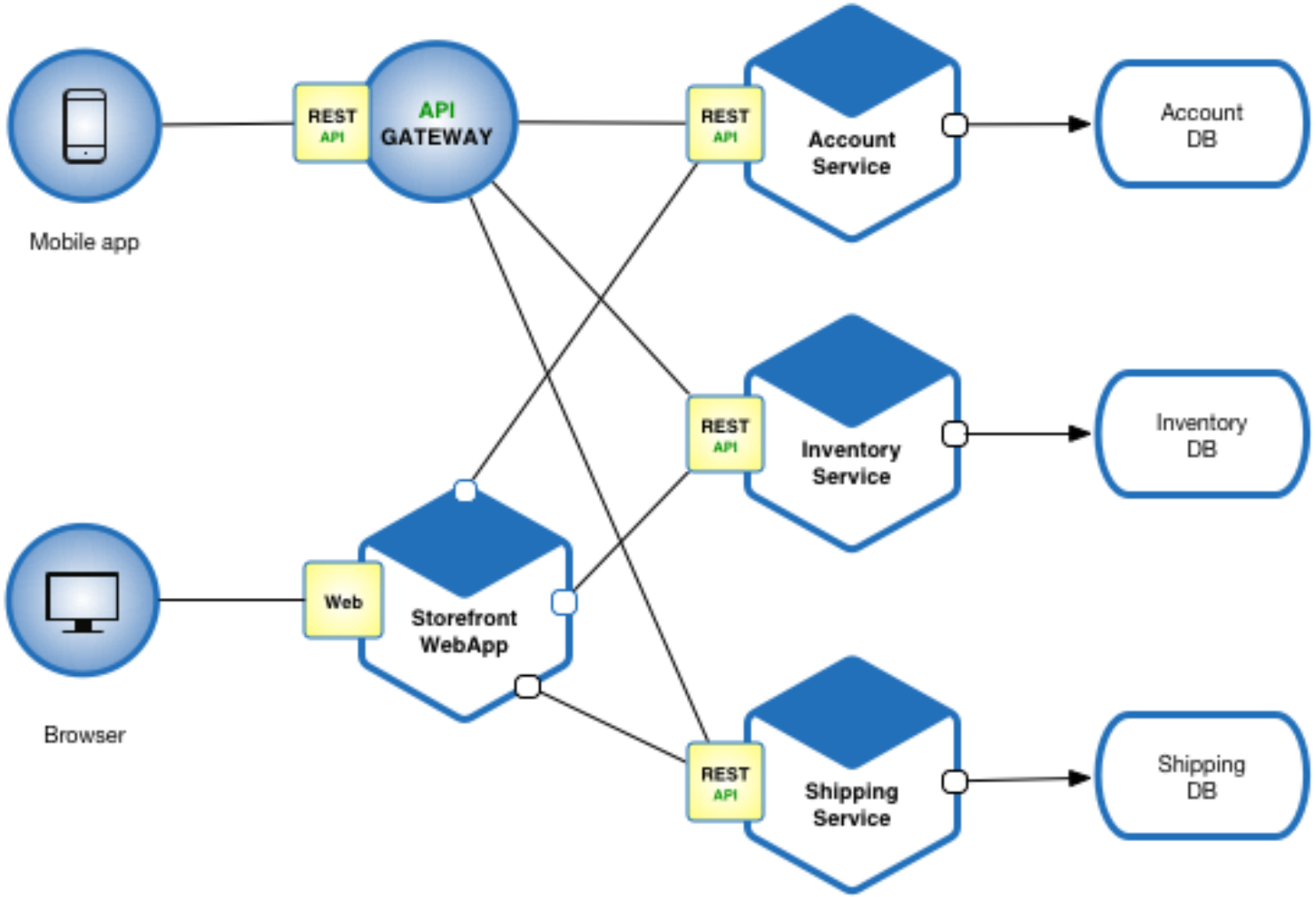
Fonte: [3]

Monolithic Application [3]

- *The application is deployed as a single monolithic application. For example, a Java web application consists of a single WAR file that runs on a web container such as Tomcat. A Rails application consists of a single directory hierarchy deployed using either, for example, Phusion Passenger on Apache/Nginx or JRuby on Tomcat. You can run multiple instances of the application behind a load balancer in order to scale and improve availability.*
  - *Implantação simplificada → EX: WAR file no TOMCAT*
- *The application consists of several components including the StoreFrontUI, which implements the user interface, along with some backend services for checking credit, maintaining inventory and shipping orders.*
  - *Arquitetura monolítica é multicamadas. Todas as camadas são implantadas juntas e formam uma aplicação única.*
- *you can scale the application by running multiple copies of the application behind a load balancer*
  - *Como escalar a aplicação → múltiplas cópias em um cluster (LB)*

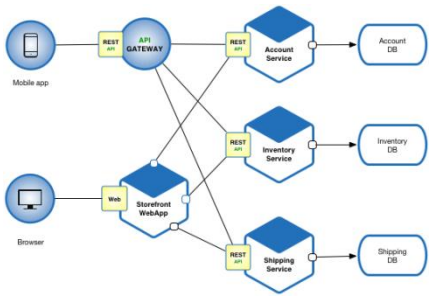


Exemplo 2 – Microservices Based



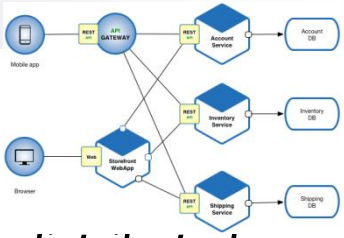
Fonte: [2]

Microservice Version [2] – Adv.



- *Each microservice is relatively small*
  - *Easier for a developer to understand*
  - *The IDE is faster making developers more productive*
  - *The application starts faster, which makes developers more productive, and speeds up deployments*
- *Each service can be deployed independently of other services - easier to deploy new versions of services frequently*
- *Easier to scale development. It enables you to organize the development effort around multiple teams.*
  - *Each (two pizza) team owns and is responsible for one or more single service. Each team can develop, deploy and scale their services independently of all of the other teams.*
- *Improved fault isolation. For example, if there is a memory leak in one service then only that service will be affected.*
  - *The other services will continue to handle requests. In comparison, one misbehaving component of a monolithic architecture can bring down the entire system.*
- *Each service can be developed and deployed independently*
- *Eliminates any long-term commitment to a technology stack. When developing a new service you can pick a new technology stack.*
  - *Similarly, when making major changes to an existing service you can rewrite it using a new technology stack.*

Microservice Version [2] – Disad.



- *Developers must deal with the additional complexity of creating a distributed system.*
  - *Developer tools/IDEs are oriented on building monolithic applications and don't provide explicit support for developing distributed applications.*
  - *Testing is more difficult*
  - *Developers must implement the inter-service communication mechanism.*
  - *Implementing use cases that span multiple services without using distributed transactions is difficult*
  - *Implementing use cases that span multiple services requires careful coordination between the teams*
- *Deployment complexity. In production, there is also the operational complexity of deploying and managing a system comprised of many different service types.*
- *Increased memory consumption. The microservice architecture replaces N monolithic application instances with NxM services instances. If each service runs in its own JVM (or equivalent), which is usually necessary to isolate the instances, then there is the overhead of M times as many JVM runtimes. Moreover, if each service runs on its own VM (e.g. EC2 instance), as is the case at Netflix, the overhead is even higher.*

Microservices VS Monolithic [1]

Table 1-1 Comparing monolithic and microservices architectures

Category	Monolithic architecture	Microservices architecture
Code	A single code base for the entire application.	Multiple code bases. Each microservice has its own code base.
Understandability	Often confusing and hard to maintain.	Much better readability and much easier to maintain.
Deployment	Complex deployments with maintenance windows and scheduled downtimes.	Simple deployment as each microservice can be deployed individually, with minimal if not zero downtime.
Language	Typically entirely developed in one programming language.	Each microservice can be developed in a different programming language.
Scaling	Requires you to scale the entire application even though bottlenecks are localized.	Enables you to scale bottle-necked services without scaling the entire application.

Microservices VS SOA [1]

- *Ver seção 1.4 livro da IBM [1]*
- *SOA possui ambições diferentes de Micros. Os serviços em SOA são projetados para serem de uso geral (por várias aplicações e pessoas). Em Micros. Um serviço é um "pedaço" de uma aplicação (antes monolítica) que foi refatorada para usar cu conjunto de microserviços que tem a intenção de servir apenas para aquela aplicação distribuída.*

Microservices VS SOA [1]

- *Em geral, microservices não precisam ser descobertos em tempo de execução e não precisam de mediação*
- *Principais diferenças das duas abordagens:*
  - *Ambição e Foco*
  - *SOA → Mais ambicioso porém mais complexo. Requer um maior investimento a priori.*
  - *Microservices → Menos custoso. Transformação mais incremental e experimental.*

Características de Microservices

- *Pequenos e Focados*
  - *Microserviços são pequenos e focam-se a resolver um problema específico de negócio*
    - *Two-Pizza Team Rule*
- *Cada Microservice é tratado como uma aplicação ou produto individual*
  - *Possui seu próprio repositório de código*
  - *Seu próprio pipeline de entrega para build e deployment*

Características de Microservices

- *Microservices são fracamente acoplados*
  - *Pode ser implantado individualmente sem depender de outros*
  - *Deployments são frequentes e rápidos*
- *Microservices são neutros quanto a linguagem*
  - *Recomenda usar a melhor linguagem de acordo com a necessidade do serviço (ex: pyhton, java, C#, etc.)*
  - *Comunicação através de APIs padronizadas (ex: REST)*

Características de Microservices

- *Bounded Context*
  - *Um Microservice não sabe nada sobre os detalhes de implementação de outros microservices*
    - *Isso não significa dizer que não podem haver dependências*

Cuidados ao adotar Microservices

- *Geralmente, não se deve começar aplicações novas já com Microservices. A idéia é você começar como uma aplicação enterprise multicamadas e a medida que começar a ficar com "cara" de monolítica, refatorar.*
- *Microservices precisam de ferramentas Devops para automatizar deployment e monitoramento. Sem isso, ficará bastante custoso.*

Cuidados ao adotar Microservices

- *Microservices podem usar suas próprias soluções de persistência, cache, message brokers, etc. Usar um PaaS pode facilitar esse trabalho.*
- *Deve-se ter cuidado para não criar um grande número de Microservices e também não torná-los pequenos demais pois terá trabalho na integração*
- *Cuidado com problemas de latência. Quando o Microservice requer muitas dependências isso pode gerar overhead.*

Cuidados ao adotar Microservices

- *In order to ensure loose coupling, each service has its own database. Maintaining data consistency between services is a challenge because 2 phase-commit/distributed transactions is not an option for many applications. An application must instead use an Event-Driven architecture. A service publishes an event when its data changes. Other services consume that event and update their data. There are several ways of reliably updating data and publishing events including Event Sourcing and Transaction Log Tailing.*
  - *Como lidar com consistência de Dados*

Escalabilidade – Scaling Cube

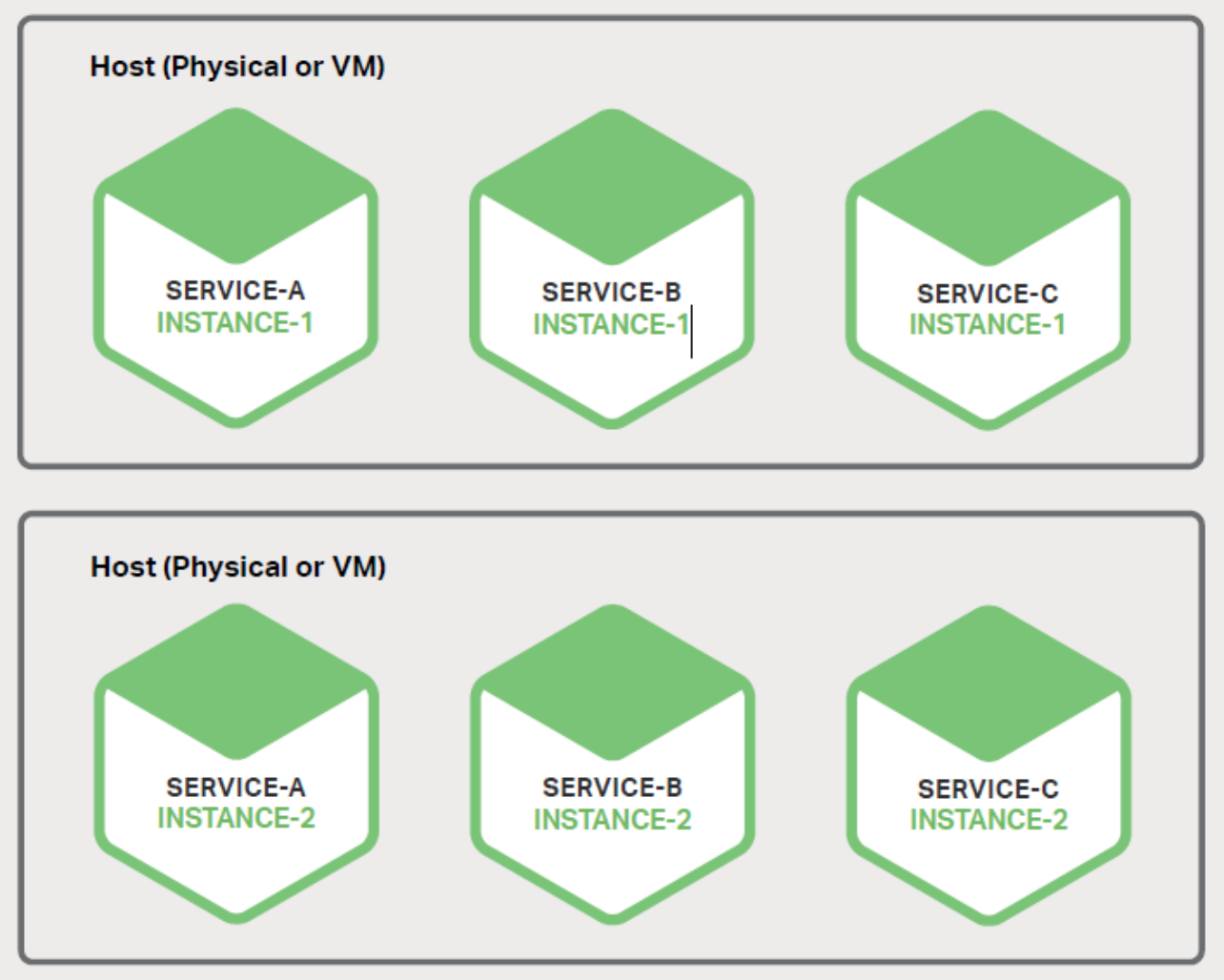
- *Ver Referência abaixo*
  - *http://microservices.io/articles/scalecube.html*
- *Microservices utiliza*
  - *Eixo X – Escalabilidade pela clonagem (criação de clusters atrás de um load balancer*
    - *Cada Microservice pode ser replicado N Vezes*
  - *Eixo Y – Escalabilidade pela decomposição funcional*
    - *Dividir a aplicação em M Microservices*
  - *Eixo Z – Particionamento dos dados*
    - *Criação de clusters que contém apenas um subconjunto dos dados (particionamento)*

Case studies

- *Ver livro da IBM seção 1.5*
  - *E-commerce site*
  - *Financial services company*
  - *Large Retailer*

Deployment Strategies: [6] capítulo 6

- *Multiple Service Instances Per Host Pattern*

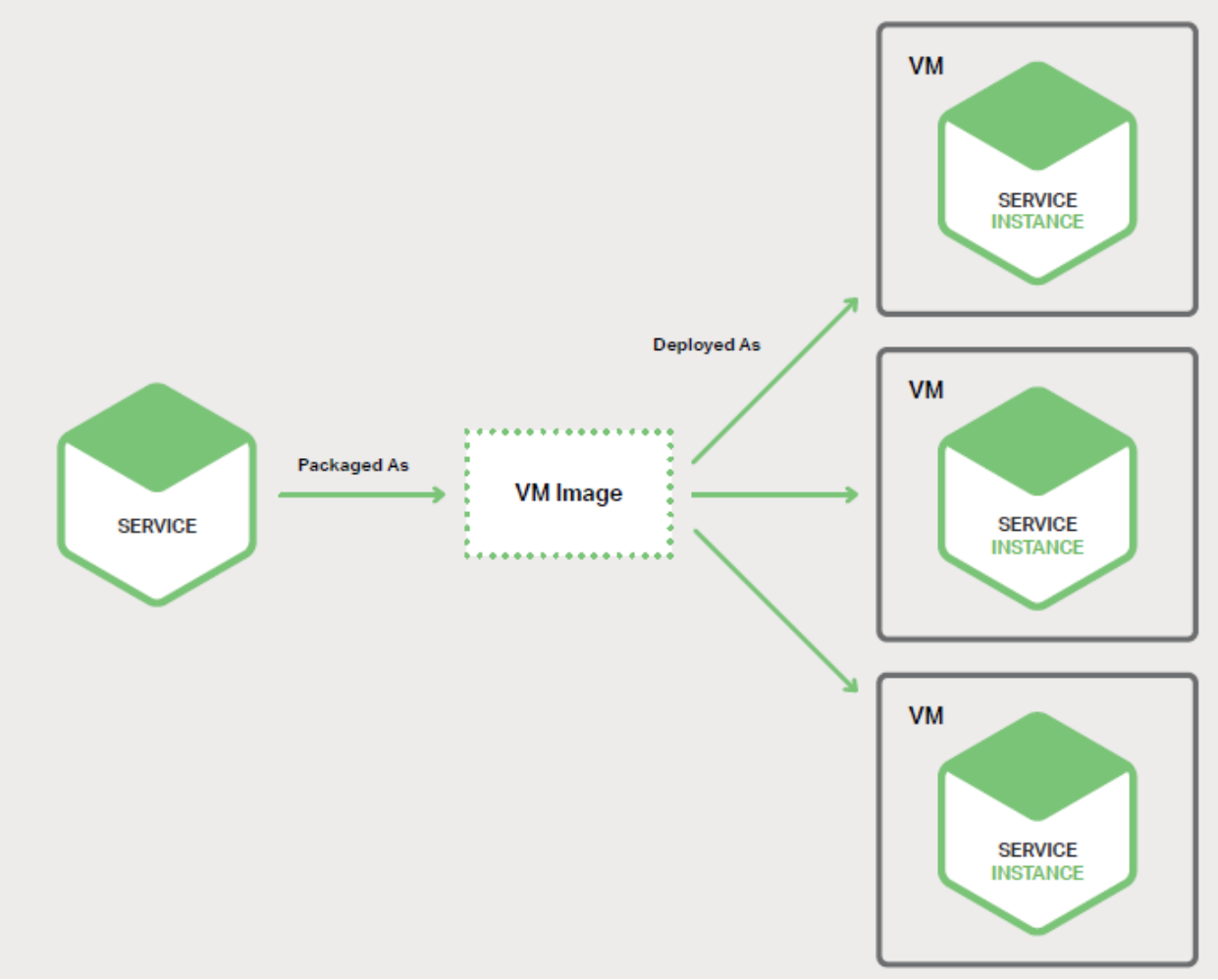


Deployment Strategies: [6] capítulo 6

- *Service Instance per Host Pattern*
  - *Service Instance per VM*
  - *Service Instance per container*

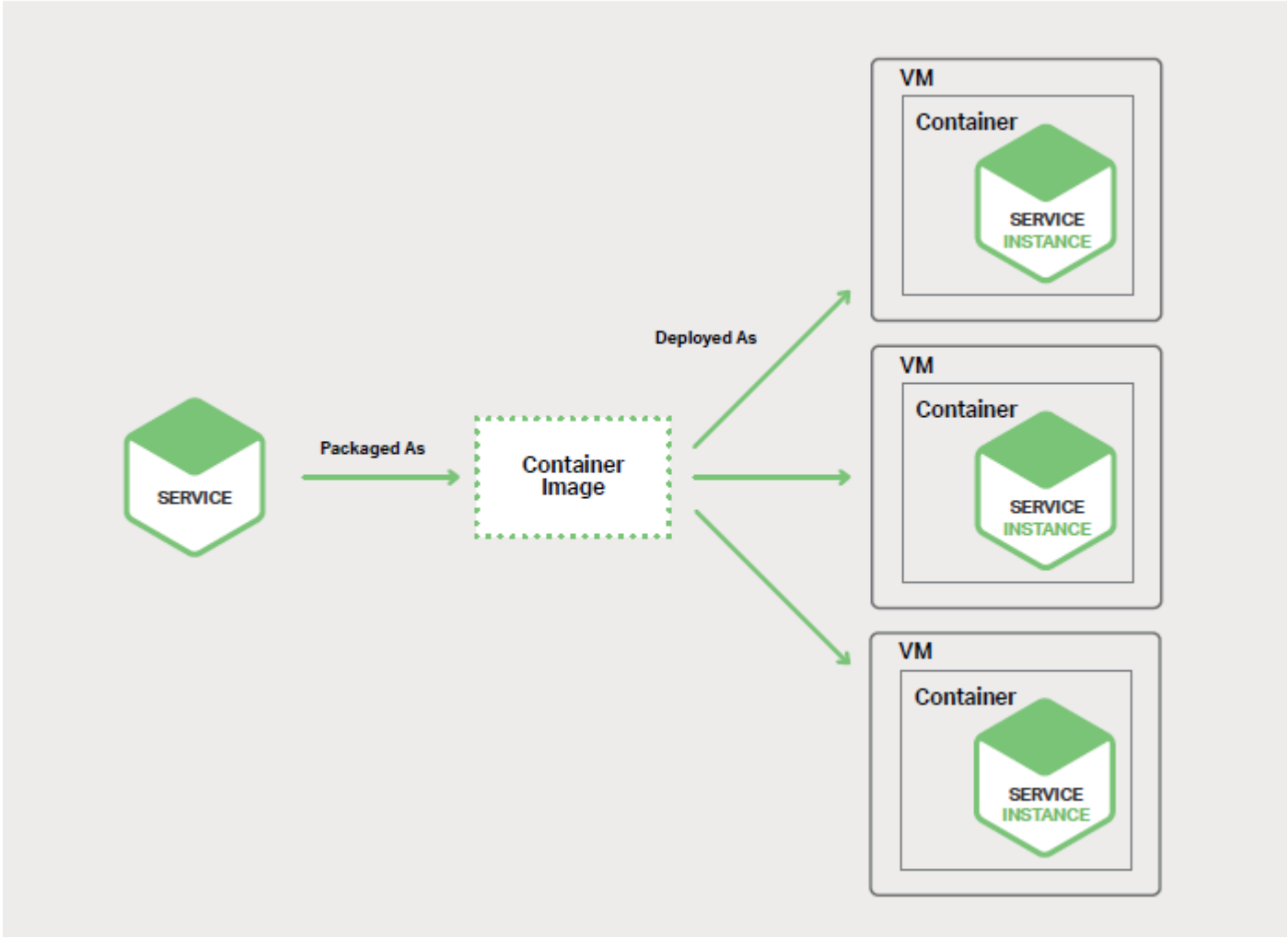
Deployment Strategies: [6] capítulo 6

- *Service Instance per VM*



Deployment Strategies: [6] capítulo 6

- *Service Instance per container*



Referências

- *[1] Microservices from Theory to Practice Creating Applications in IBM Bluemix Using the Microservices Approach. IBM Reedbook. 2015.*
- *[2]* Chris Richardson http://microservices.io/patterns/microservices.html
- *[3]* Chris Richardson http://microservices.io/patterns/monolithic.html
- *[4] Martin Fowler* https://martinfowler.com/articles/microservices.html
- *[5] Martin Fowler.* https://martinfowler.com/microservices/
- *[6] Chris Richardson and Floyd Smith. Microservices: From Design to Deployment.*